

[University Name]

[Department Name]

FINAL YEAR PROJECT REPORT

**Real-Time Cyber-Attack Detection for Industrial
Control Systems in a Water Treatment Plant**

*A Protocol-Aware and Deep-Learning Based Detection-in-Depth Framework with
Tamper-Evident Logging*

Submitted by

[Member Name 1] ([Roll No.])

[Member Name 2] ([Roll No.])

[Member Name 3] ([Roll No.])

[Member Name 4] ([Roll No.])

Under the supervision of

[Supervisor Name]

[Designation, Department]

[Month Year]

Abstract

Industrial control systems (ICS) that run critical infrastructure such as water treatment communicate over protocols like Modbus that have no authentication, so an attacker on the control network can falsify sensor readings or issue unauthorised commands. This project designs, implements and evaluates, entirely in software, a detection-in-depth framework for a simulated five-stage water treatment plant. The plant is modelled in Python and exchanges genuine Modbus TCP traffic with a supervisory controller and an attacker.

Three detection layers run in parallel: a protocol-aware deep packet inspection (DPI) rule engine that decodes Modbus writes and produces human-readable alerts; a flow-volume monitor that detects request floods; and an LSTM autoencoder, trained only on normal operation, that detects abnormal process behaviour. Every control command and alert is recorded in a hash-chained, digitally signed ledger that exposes any later alteration, and an operator dashboard displays plant state, alerts and ledger integrity.

On a labelled 510-second dataset containing five attack types, no single layer flags more than 60% of attack seconds, while the combined system flags all of them (recall 1.00, precision 0.89) and raises an alert in the first second of every attack. All combined false positives occur in the nine seconds after an attack ends, where the LSTM's ten-second window still overlaps attack data. To test whether the learning-based layer generalises, it was also trained and tested on the public HAI 22.04 dataset from a real hardware-in-the-loop testbed. There it ranked attack windows above normal ones better than chance (ROC-AUC 0.74) but did not provide a usable alarm: with the threshold set on a held-out normal recording it detected 2 of 7 attacks at a 0.1% false-positive rate. The main cause is that the plant's operating mode changes between recordings.

The results support layered detection for ICS, and show that the learning-based layer needs mode-aware normalisation before it can be relied on in a real plant.

Keywords: industrial control systems; Modbus; intrusion detection; deep packet inspection; LSTM autoencoder; tamper-evident logging; water treatment.

Table of Contents

Abstract	i
List of Abbreviations	vi
1 Introduction.....	1
1.1 Background and Motivation	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.4 Scope and Constraints.....	2
1.5 Report Organisation	3
2 Literature Review.....	4
2.1 Threats to ICS Communication	4
2.2 Protocol-Aware Network Intrusion Detection	4
2.3 Automated Detection-in-Depth.....	4
2.4 Deep Learning for Sequence Anomaly Detection	5
2.5 Tamper-Evident Logging.....	5
2.6 Research Gap	5
3 System Design	6
3.1 Design Principles	6
3.2 Architecture.....	6
3.3 Attack Model	7
3.4 Evaluation Method.....	8
4 Implementation	10
4.1 Development Environment	10
4.2 Plant Simulator.....	10
4.3 Dataset Generation.....	11
4.4 Protocol-Aware DPI Rule Engine.....	12
4.5 Flow-Volume Monitor	13
4.6 LSTM Autoencoder	13
4.7 Correlator and Tamper-Evident Ledger.....	14
4.8 Operator Dashboard	15
4.9 Testing.....	16
5 Results.....	17
5.1 Detection by Each Layer.....	17
5.2 Ablation.....	17
5.3 Recall per Attack Class.....	18
5.4 Detection Latency	19

5.5 External Validation on HAI 22.04	19
6 Discussion and Limitations.....	23
6.1 What the Results Show	23
6.2 Limitations	23
6.3 Lessons Learned.....	24
7 Conclusion and Future Work.....	25
7.1 Conclusion	25
7.2 Status of the Objectives	25
7.3 Future Work.....	25
References.....	27
Appendix A Reproducing the Results.....	28
Appendix B Register Map	29
Appendix C Automated Tests.....	30

List of Figures

Figure 1: System architecture of the detection-in-depth framework	7
Figure 2: Generated dataset: tank level, pH and two pump states over 510 s, with the five attack windows shaded	12
Figure 3: Operator dashboard during a false data injection on LIT101, with the audit ledger shown as intact.....	15
Figure 4: Audit-ledger panel after a stored entry was altered, showing the failed verification	16
Figure 5: Per-second precision, recall and false-positive rate of each layer and of the combined system.....	18
Figure 6: Recall per attack class for each layer	19
Figure 7: HAI round 1: worst-signal anomaly score over the test day, with attacks shaded; the threshold from training data lies below most of the day	21
Figure 8: HAI round 2: worst-signal score after training on three recordings; the large drop near 16.8 h is a change of operating mode, not an attack.....	22

List of Tables

Table 1: Substitutions relative to the project proposal.....	2
Table 2: Attack scenarios implemented in the simulator.....	8
Table 3: Software used.....	10
Table 4: Contents of the generated dataset	11
Table 5: DPI rules, in the order they are checked.....	13
Table 6: LSTM window detection with two scoring methods (simulator, same trained model)	14
Table 7: Detections by each layer in its own unit of analysis.....	17
Table 8: Per-second ablation on the simulator dataset.....	17
Table 9: Per-second recall by attack class	18
Table 10: Seconds from the start of each attack to the first alert.....	19
Table 11: HAI 22.04 files used	20
Table 12: LSTM results on HAI test1 (window level; random PR-AUC = 0.011).....	20
Table 13: Objectives from Section 1.3 and their status	25
Table 14: Plant register map	29
Table 15: Automated tests	30

List of Abbreviations

Abbreviation	Meaning
CIP	Common Industrial Protocol
DoS	Denial of Service
DPI	Deep Packet Inspection
ENIP	EtherNet/IP
FDI	False Data Injection
FPR	False-Positive Rate
HAI	HIL-based Augmented ICS (security dataset)
HIL	Hardware-in-the-Loop
HMI	Human–Machine Interface
ICS	Industrial Control System
LSTM	Long Short-Term Memory
MSE	Mean Squared Error
NIDS	Network Intrusion Detection System
OT	Operational Technology
PLC	Programmable Logic Controller
PR-AUC	Area Under the Precision–Recall Curve
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SCADA	Supervisory Control and Data Acquisition
SWaT	Secure Water Treatment (testbed)
TCP	Transmission Control Protocol

1 Introduction

1.1 Background and Motivation

Industrial control systems (ICS) operate the physical processes behind water treatment, power generation, oil and gas, and manufacturing. A plant is typically controlled by programmable logic controllers (PLCs) that read sensors and drive actuators such as pumps and valves, supervised by a human–machine interface (HMI) or SCADA system. As these operational technology (OT) networks are connected to corporate IT networks and the internet, they gain remote monitoring and analytics but also become reachable by attackers.

The protocols used on OT networks, such as Modbus, EtherNet/IP (ENIP) and DNP3, were designed for deterministic, low-latency control on isolated networks. They generally carry no authentication or encryption: a device accepts any well-formed command from any host that can reach it. An attacker with network access can therefore inject false sensor values, replay previously captured commands, or command actuators directly. Real incidents show that these threats are practical. Stuxnet altered PLC logic while showing operators normal-looking process data; the attacks on the Ukrainian power grid in 2015 and 2016 opened breakers and cut power; and intrusions at water facilities in Oldsmar, Florida (2021) and Muleshoe, Texas (2024) involved remote access to plant controls.

Water treatment is a natural domain in which to study these threats. It is a multi-stage physical process (raw water intake, chemical dosing, ultrafiltration, dechlorination and reverse osmosis) in which a manipulated reading or command has a direct public-health consequence, and it has well-documented reference testbeds such as SWaT [7] and HAI [3].

1.2 Problem Statement

Design and implement a detection-in-depth cybersecurity framework for a simulated water treatment ICS that (a) detects known data-manipulation attack patterns using protocol-aware deep packet inspection, (b) detects stealthy or previously unseen attacks using a sequence anomaly detector trained on normal process behaviour, and (c) keeps a tamper-evident record of control actions and alerts, so that plant operators can trust, verify and act on detection results. The framework must be built and evaluated without physical ICS hardware.

1.3 Objectives

1. Simulate a representative multi-stage water treatment process with PLC-style control of its sensors and actuators.
2. Generate OT network traffic and device logs for normal operation and for data-manipulation, replay and flooding attacks.
3. Implement a protocol-aware rule-based detector that identifies the targeted device, requested action and injected value, and produces human-readable alerts.
4. Implement an LSTM-based anomaly detector that learns normal temporal behaviour to catch attacks that evade the rule-based layer.
5. Record control commands and alerts in a tamper-evident ledger.
6. Provide an operator dashboard showing plant status, alerts and log integrity.
7. Evaluate each layer and the combined system using precision, recall, false-positive rate and detection latency, and test generalisation on a public dataset.

1.4 Scope and Constraints

The project was carried out without physical ICS hardware. Every component (plant, controller, network traffic, attacker and detectors) runs as software on a single Windows laptop. This required the substitutions in Table 1, each chosen so that the part of the system under study is still exercised with realistic inputs.

Table 1: Substitutions relative to the project proposal

Proposal item	Implemented as	Reason
Physical testbed / SWaT hardware	Python physics model of five coupled treatment stages	No hardware available; stages are coupled so a change in one propagates downstream
EtherNet/IP (ENIP/CIP) traffic	Modbus TCP	Same weaknesses (no authentication, write-by-address); mature open-source tooling
MATLAB/Simulink, OpenPLC	Python process model and Python controller	Simulink adds nothing the detectors consume; OpenPLC was prepared but not integrated
Hyperledger Fabric	Signed SHA-256 hash chain behind a ledger interface	Gives tamper evidence; Fabric can replace it behind the same interface
SWaT dataset	Own labelled dataset plus public HAI 22.04	SWaT requires an access application; HAI is openly available
FastAPI/Flask + PostgreSQL dashboard	FastAPI + WebSocket dashboard, ledger in memory or a file	PostgreSQL was not needed for the experiments

Because the attacks and operating conditions in the simulator were designed by the project team, the simulator results show how the detection design behaves under controlled conditions. The validation on the public HAI dataset (Section 5.5) is the test against data from a real process.

1.5 Report Organisation

Chapter 2 reviews related work. Chapter 3 describes the system design, the attack model and the evaluation method. Chapter 4 describes how each component was implemented. Chapter 5 presents the results on the simulator dataset and on HAI. Chapter 6 discusses limitations and lessons learned, and Chapter 7 concludes and outlines future work.

2 Literature Review

2.1 Threats to ICS Communication

Modbus is a request–response protocol in which a client reads or writes numbered registers and coils on a server device [5]. Each request carries a function code (for example 3 to read holding registers and 6 to write a single register), a register address and, for writes, a value. The protocol has no field for identifying or authenticating the sender, so a server acts on any well-formed request. Five attack classes follow directly from this: false data injection (overwriting a sensor value so operators and controllers see a false reading), command injection (issuing an actuator command from an unauthorised host), replay (re-sending a previously captured valid command), flooding (sending requests fast enough to degrade the control loop), and stealthy manipulation, in which the attacker's commands look valid at the protocol level and the attack shows only in how the process behaves.

2.2 Protocol-Aware Network Intrusion Detection

Gauthama Raman et al. [1] proposed PA-NIDS, a rule-based, protocol-aware intrusion detection system that performs deep packet inspection of ENIP/CIP traffic sent to PLCs. Rather than using abstract statistical features, it decodes the Instance Identifier, Member Identifier and Command-Specific Data fields inside Write_Tag_Service commands to determine which device is targeted, what action is requested and what value is being injected. On the SWaT testbed it detected all five tested single-point and multi-point attacks with no false positives and a detection latency under five seconds, much faster than two process-based anomaly detectors it was compared with. Its main strength is interpretability: every alert names the targeted tag, the likely intent and the source and destination addresses. Its main limitations are that device-specific identifiers must be mapped by hand, and that it only detects attacks that appear as explicit protocol write commands; attacks that leave protocol fields untouched, or unfold as gradual process drift, are outside its reach.

2.3 Automated Detection-in-Depth

Jadidi et al. [2] proposed AFAD, a two-layer detection-in-depth architecture. One layer clusters histograms of NetFlow traffic features using hierarchical cluster analysis; the other forecasts device-log time series with an ARIMA/GARCH model. Together they detect network-level flooding and physical-layer manipulation such as man-in-the-middle attacks. Evaluated on a factory-automation dataset, a Modbus dataset and SWaT, AFAD reached 0.96 precision and 0.91 recall on the factory dataset and 0.92 precision and 0.91 recall on

SWaT, about ten percentage points better in precision than single-source detectors. Both layers are unsupervised, which avoids the cost of labelling ICS data. Its limitations, acknowledged by the authors, are that it reports an anomaly without identifying the type or intent of the attack, and that its histogram extraction step is manual.

2.4 Deep Learning for Sequence Anomaly Detection

Long short-term memory (LSTM) networks [6] are recurrent neural networks that can model long-range dependencies in time series. LSTM encoder–decoder models trained to reconstruct normal multivariate sequences [8] are widely used for anomaly detection: a window that the model reconstructs poorly is treated as anomalous. Because they need only normal data for training, they suit ICS, where labelled attack data is scarce. Their practical difficulties are choosing an alarm threshold and coping with changes in normal operating conditions, which can look as unusual to the model as an attack does. Both difficulties appear in this project's results on real data (Section 5.5).

2.5 Tamper-Evident Logging

An attacker who has compromised a plant may try to erase or alter the evidence. Hash chains address this by linking each log record to the previous one through a cryptographic hash, so that changing any record changes every later hash. Blockchains such as Hyperledger Fabric add replication and consensus across several parties. Digital signatures such as Ed25519 [9] add a further guarantee: an attacker who alters a record and recomputes the chain still cannot produce valid signatures without the signing key. Within a single organisation, a signed hash chain provides tamper evidence without the operational cost of running a blockchain network.

2.6 Research Gap

The two reference systems are complementary. PA-NIDS is interpretable and fast but protocol-specific and rule-based, so it cannot detect attacks that do not appear as explicit writes. AFAD generalises across data sources using unsupervised learning, but its alerts carry no information about the attacker's intent, and it does not decode protocol semantics. Neither provides tamper-evident logging of detection events, which matters for forensic investigation in regulated industries. This project combines protocol-aware, interpretable rule-based detection with a volumetric flow monitor and a learned sequence model, adds tamper-evident logging, and measures the combination against each layer on its own.

3 System Design

3.1 Design Principles

- **Detection in depth.** Each layer targets attacks the others cannot see, and an alert from any layer is reported.
- **Interpretable alerts.** Where possible, an alert names the device, the action, the value and the source, following PA-NIDS.
- **No attack labels for training.** The learned components (the flow monitor's threshold and the LSTM) are trained on normal operation only, because labelled attack data is scarce in real plants.
- **Tamper evidence.** Every control command and alert is written to a ledger whose integrity can be checked later.
- **Real protocol traffic.** The controller, the attacker and the plant exchange genuine Modbus TCP requests.

3.2 Architecture

Figure 1 shows the architecture. The plant simulator exposes its sensors and actuators as Modbus holding registers. A Python controller acting as the PLC/HMI reads the registers and writes actuator commands over Modbus TCP, and an attacker host sends its own requests to the same server. Two data streams are recorded. The **network log** holds every Modbus request, decoded into source and destination address, function code, register and value. The **device log** is a once-per-second history of every sensor reading and actuator state, as a plant historian would record it. The DPI rule engine and the flow monitor read the network log; the LSTM reads the device log. A correlator merges their alerts into one time-ordered stream, writes each alert and each legitimate control command to the ledger, and supplies the dashboard.

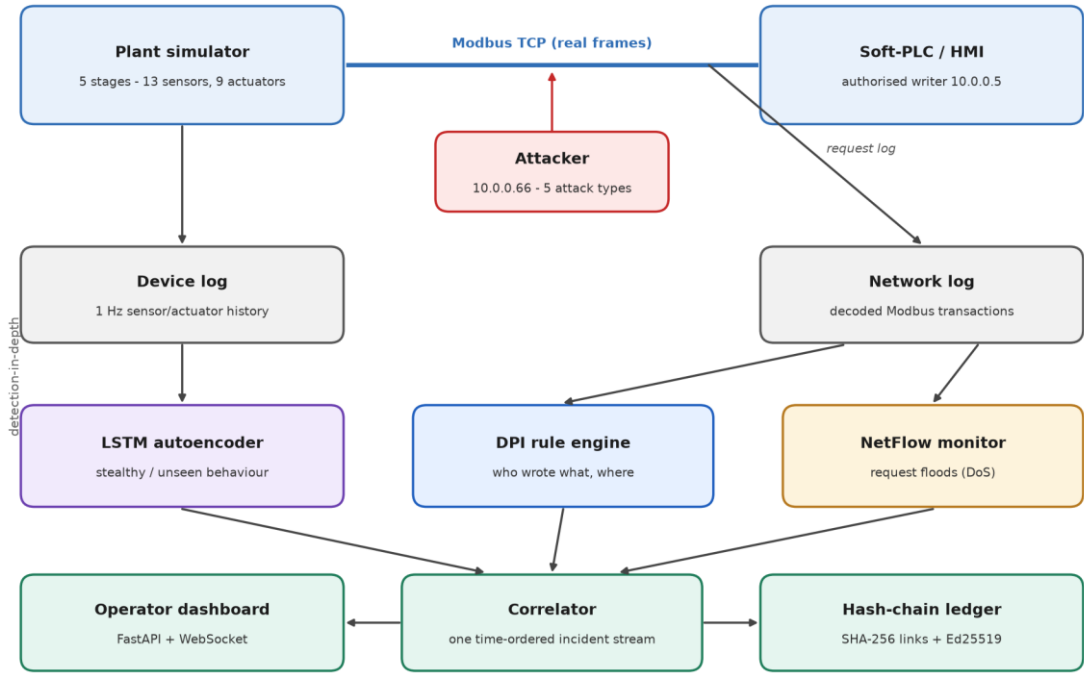


Figure 1: System architecture of the detection-in-depth framework

3.3 Attack Model

The attacker is assumed to have network access to the OT segment and to be able to send arbitrary Modbus requests. In the stealth scenario the attacker is also assumed to control the HMI, and so can send commands from the authorised address. The attacker cannot modify the detectors or obtain the ledger's signing key. Table 2 lists the five attacks implemented.

Table 2: Attack scenarios implemented in the simulator

Attack	What the attacker does	Where it is visible	Expected layer
False data injection	Writes 1180 mm to the raw-water tank level LIT101 (safe limit 1100 mm) every second, so the HMI reads a false value	Write to a sensor register from an unknown host	DPI, LSTM
Command injection	Forces the raw-water transfer pump P101 off every second from an unknown host	Unauthorised actuator write; process change	DPI, LSTM
Replay	Re-sends the controller's most recent recorded write from the attacker's address	Write from an unauthorised address	DPI
Flooding (DoS)	Sends 40 read requests per second to the PLC	Traffic volume only	Flow monitor
Stealth manipulation	Holds the HCl dosing pump P203 on using the HMI's own address, a valid command that drives the pH down	Process behaviour only	LSTM

3.4 Evaluation Method

The simulator results are evaluated per second. The ground truth for each second is the label in the device log: the second is an attack second if it lies inside an attack window. Each layer's alerts are attributed to the second in which they fire: for DPI and the flow monitor this is the second of the triggering request; for the LSTM it is the last second of the ten-second window being scored. The following metrics are reported:

- **Precision** = $TP / (TP + FP)$: the fraction of alerted seconds that were attack seconds.
- **Recall** = $TP / (TP + FN)$: the fraction of attack seconds that were alerted.
- **False-positive rate (FPR)** = $FP / (FP + TN)$: the fraction of normal seconds that were alerted.
- **F1**, the harmonic mean of precision and recall, and **recall per attack class**.
- **Detection latency**: the number of seconds from the start of an attack episode to the first alert within it.

An ablation compares each layer on its own with the combined system, in which a second counts as alerted if any layer alerts in it. For the external validation on HAI, which has one label per second but no network traffic, the LSTM is evaluated on ten-second windows, a window counting as an attack window if any of its seconds is an attack second. Because attacks cover only about 1% of HAI's test data, threshold-free ranking metrics (ROC-AUC

and PR-AUC) are reported alongside precision, recall and FPR at the deployable threshold, and alongside the best F1 achievable if the threshold were chosen using the test labels (an upper bound that cannot be achieved in practice).

4 Implementation

4.1 Development Environment

The system is written in Python and kept in a private Git repository with 24 automated tests. Table 3 lists the main libraries.

Table 3: Software used

Purpose	Tool (version)
Language	Python 3.13.2
Modbus TCP server and clients	pymodbus 3.11.3
LSTM autoencoder	PyTorch 2.14 (CPU) [10]
Data handling and metrics	NumPy 2.5, pandas 3.0, scikit-learn 1.9
Dashboard	FastAPI 0.141 with WebSocket, served by Uvicorn 0.52
Ledger signatures	cryptography 50.0 (Ed25519)
Plots	matplotlib 3.11
Tests	pytest 9.1

4.2 Plant Simulator

Register map. The plant has 22 points: 13 sensors and 9 actuators, named after the SWaT convention (for example LIT101 for the raw-water tank level, AIT202 for pH after dosing, P101 for the raw-water transfer pump and MV301 for the ultrafiltration backwash valve). Each point has a Modbus address, an engineering unit, a scale factor (registers are 16-bit integers, so a pH of 7.35 is stored as 735) and a safe operating range. The register map is the single definition shared by the simulator, the controller and the DPI rule engine. The full map is given in Appendix B.

Process model. The model advances in one-second steps. Tank levels change by the difference between inflow and outflow divided by the tank area. The NaOCl and HCl dosing pumps raise and lower the pH, which otherwise relaxes towards 7.3; conductivity follows the pH. The ultrafiltration differential pressure rises with throughput and falls when the backwash valve is open, and reverse-osmosis pressure and permeate flow follow the high-pressure pump. The stages are coupled: the outflow of one stage is the inflow of the next, so a change in one actuator propagates downstream. Sensor readings carry 1% Gaussian noise.

Control logic. The controller implements simple band control. For example, the inlet valve MV101 closes when LIT101 exceeds 900 mm, the transfer pump P101 runs while LIT101 is above 500 mm, the NaOCl pump runs when the pH falls below 6.8 and the HCl pump when it rises above 7.8, and the ultrafiltration unit is backwashed when its differential pressure exceeds 35 kPa.

Modbus server. A pymodbus asynchronous TCP server serves the plant state as holding registers, so any Modbus client (the controller, the attacker or the dashboard) reads and writes the plant through the real protocol.

4.3 Dataset Generation

The dataset generator runs the plant, the controller and the attacker together, one simulated second at a time. In each second: (1) the plant reads its actuator registers and advances one step; (2) it publishes noisy sensor values; (3) the attacker performs any sensor-spoofing write, so that the controller will read the spoofed value; (4) the controller reads the whole register block over Modbus TCP and re-sends any actuator command whose register does not hold the value its control logic wants; (5) the attacker performs any actuator write, flood or replay; and (6) the device log records the reported sensor values and the final actuator states.

Each actor has a logical address on a simulated OT segment: the PLC at 10.0.0.2, the HMI at 10.0.0.5 and the attacker at 10.0.0.66. The generator records every Modbus request it sends, with the fields a packet decoder would extract, and tags it with the simulated second and the ground-truth label. The network log is therefore produced by the traffic generator rather than by capturing packets; capturing the frames with a tool such as tshark would give the same records but was not implemented.

The default run lasts 510 seconds: 60 seconds of normal operation, then each of the five attacks for 45 seconds, separated by 45 seconds of normal operation (Table 4 and Figure 2).

Table 4: Contents of the generated dataset

Log	Rows	Breakdown
Device log (one row per second, 22 signals)	510	285 normal; 45 per attack class (225 attack seconds)
Network log (one row per Modbus request)	2,608	628 normal; 1,800 flooding; 45 each for false data injection, command injection, replay and stealth

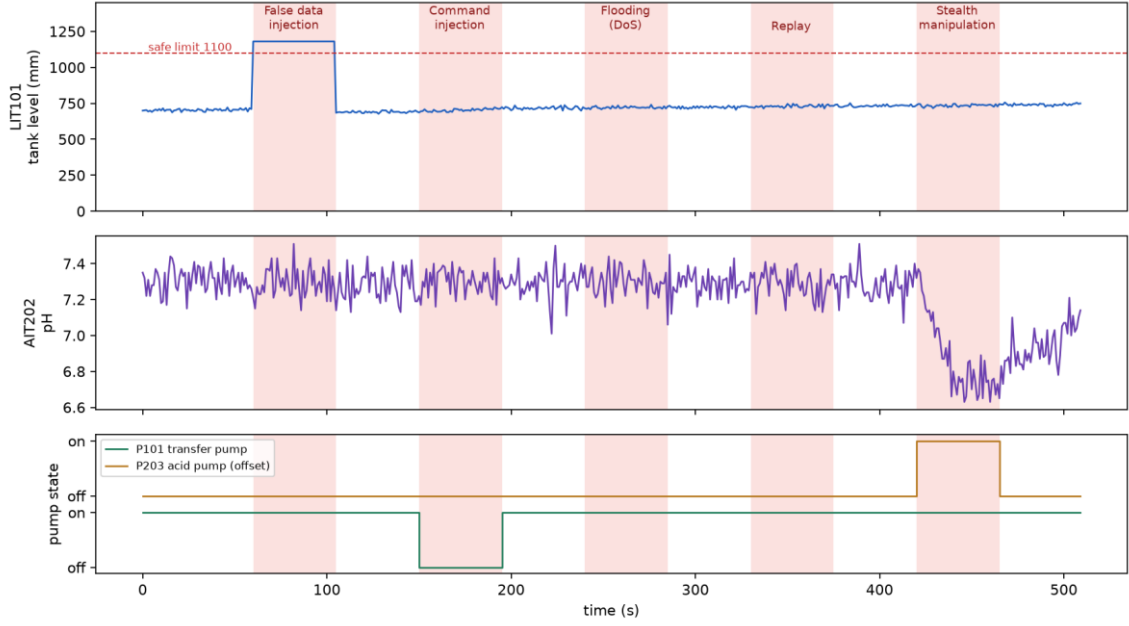


Figure 2: Generated dataset: tank level, pH and two pump states over 510 s, with the five attack windows shaded

A data error found and fixed. An earlier version of the generator had the controller send a command only when its own decision changed. It therefore sent each pump command once, at the start, and never again. After the command-injection and stealth attacks it never switched the affected pump back, so seconds labelled normal after those attacks still carried the attack's effect, and the LSTM was partly trained on them. The error was found while plotting the dataset for this report. The controller now compares its desired output with the register it has just read and re-sends the command on any mismatch, as a PLC re-applies its outputs every scan cycle. A test now checks that pumps are restored when an attack ends; it fails on the old code and passes on the new. All results in Chapter 5 come from the corrected dataset.

4.4 Protocol-Aware DPI Rule Engine

The DPI engine inspects every write request (Modbus function codes 6 and 16) and applies the rules in Table 5 in order; the first rule that matches produces the alert. Read requests are not inspected by these rules. The list of sources allowed to write contains only the HMI address.

Table 5: DPI rules, in the order they are checked

Rule	Condition	Detects	Severity
sensor_write	A write targets a sensor register (sensors are plant outputs and should never be written)	False data injection	High
unauthorized_writer	A write comes from a source that is not on the authorised-writer list	Command injection, replay	High
out_of_band_setpoint	An authorised write sets an actuator to an illegal state or a value outside its safe range	Set-point manipulation	Medium

Each alert is a complete sentence an operator can act on. For the command-injection attack, the engine produces: “Unauthorised source 10.0.0.66 issued a WRITE to P101 (Raw water transfer pump) = 0.0 state, targeting 10.0.0.2. No such source is permitted to command actuators - likely command injection.”

The engine has two deliberate blind spots. It does not flag reads, so a flood of read requests passes through; and it does not flag a valid command from the authorised address, so a compromised HMI passes through. These are the gaps the other two layers are designed to cover.

4.5 Flow-Volume Monitor

The flow monitor counts requests per source per second, a simple flow feature in the spirit of AFAD's NetFlow layer. It is fitted on normal traffic only: its threshold is the larger of the normal mean plus eight standard deviations and 1.5 times the largest normal count. On the generated dataset the busiest normal source sent at most 3 requests per second (mean 1.23), giving a threshold of 5; the flooding attacker sends 40. Each alert names the source and its request count.

4.6 LSTM Autoencoder

Input. The model scores ten-second windows of all 22 signals, standardised using the mean and standard deviation of normal seconds.

Architecture and training. An encoder LSTM (22 inputs, 32 hidden units) compresses the window, a linear layer maps the final hidden state to a 16-dimensional code, and a decoder LSTM expands the code back into a ten-step sequence of 22 signals. The model was trained for 80 epochs with the Adam optimiser (learning rate 0.001) and mean-squared reconstruction loss, on the 231 windows that contain only normal seconds.

Scoring. For each window the reconstruction error of each signal is averaged over time and standardised by that signal's error mean and standard deviation on the training windows; the window's score is the largest of these standardised errors. The alarm threshold is the larger of the 99.5th percentile and 1.05 times the maximum score on the training windows (15.89). Each alert names the signal with the largest standardised error.

Taking the worst signal, rather than averaging the error over all signals, is what allows the model to catch the stealth attack, which changes mainly one pump and the pH. Table 6 compares the two scores on the same trained model with the same threshold rule.

Table 6: LSTM window detection with two scoring methods (simulator, same trained model)

Windows	Mean MSE over all signals	Worst standardised signal (used)
False data injection	54 / 54	54 / 54
Command injection	53 / 54	54 / 54
Stealth manipulation	29 / 54	54 / 54
Flooding, replay	0 / 108	0 / 108
Normal (false positives)	0 / 231	0 / 231

On HAI the ranking is reversed (Section 5.5): with 60 or more noisy real sensors, the worst-signal score is more easily triggered by noise and by changes in operating mode.

4.7 Correlator and Tamper-Evident Ledger

The correlator runs the three layers over the recorded logs, orders their alerts by second, and appends every legitimate control command and every alert to the ledger. Each ledger entry stores its index, a timestamp, the hash of the previous entry, the payload and the payload's hash. The entry hash is SHA-256 over the index, the previous hash, the payload hash and the timestamp, and the entry hash is signed with an Ed25519 key. Verification walks the chain and checks, for every entry, the index, the link to the previous entry, the payload hash, the entry hash and the signature, and reports the first entry that fails. The ledger sits behind an interface so that a Hyperledger Fabric backend could replace it without changes elsewhere.

On the generated dataset the correlator produced 342 alerts and a ledger of 505 entries, which verified as intact. Changing the payload of entry 5 produced “COMPROMISED - break at entry 5: payload altered (hash mismatch)”. Automated tests confirm that editing,

deleting and reordering entries, and re-signing an altered entry with a different key, are all detected.

4.8 Operator Dashboard

The dashboard is a FastAPI application that runs the plant in the same process and pushes a snapshot of all registers to the browser over a WebSocket every second (Figure 3). Sensor cards turn red when a reading leaves its safe range, and actuator states are shown alongside. An **Inject attack** button starts a sustained false data injection on LIT101 from a separate client connection. An audit-ledger panel shows the entries recorded during the session, with an INTACT or COMPROMISED indicator, and a **Tamper with log** button alters a stored entry to demonstrate that the change is detected (Figure 4). The dashboard's own live alerting is a safe-range check used for demonstration; the three detection layers run through the correlator over the recorded logs.

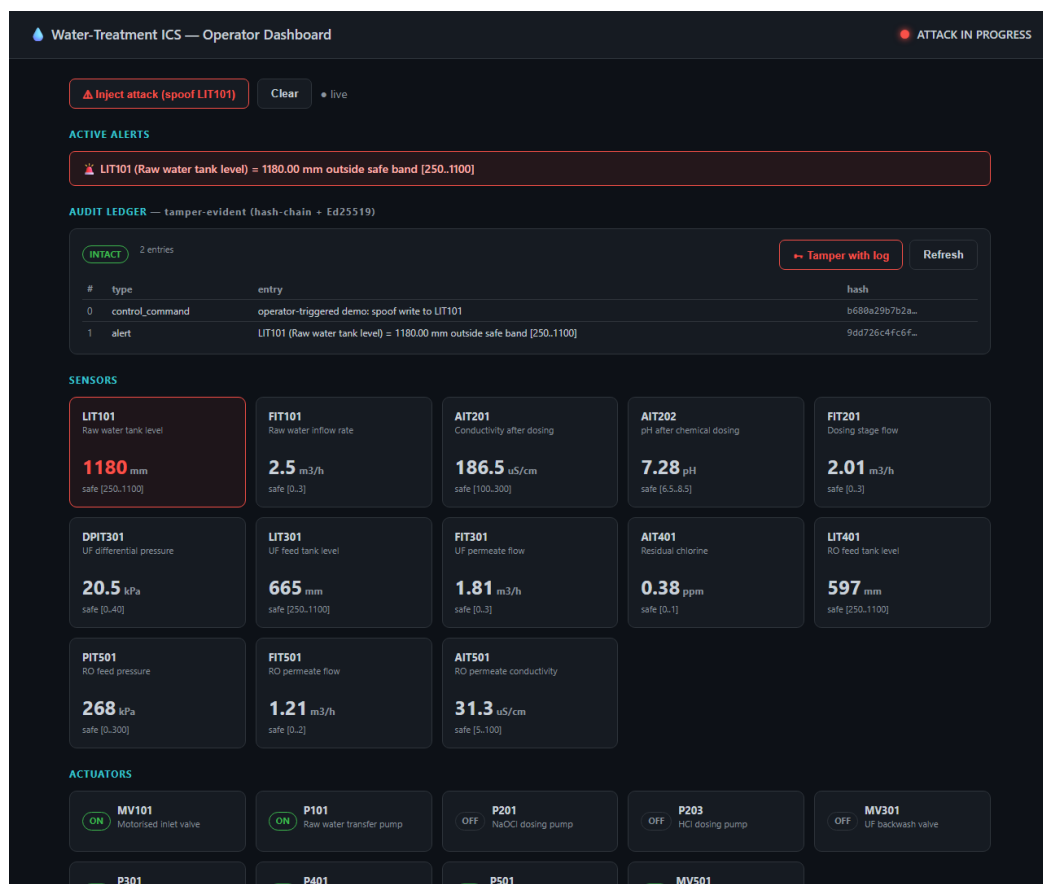


Figure 3: Operator dashboard during a false data injection on LIT101, with the audit ledger shown as intact

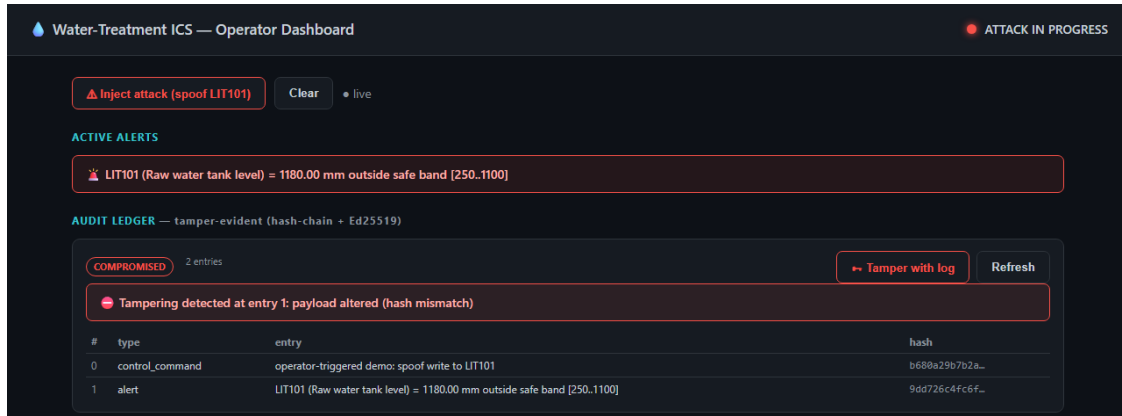


Figure 4: Audit-ledger panel after a stored entry was altered, showing the failed verification

4.9 Testing

The 24 automated tests (Appendix C) cover the Modbus round trip, the dataset generator (including restoration of actuators after an attack), each DPI rule and its intended blind spots, the flow monitor, the LSTM scoring mechanism, every ledger tampering case, the evaluation metrics and the HAI helper functions. All 24 pass.

5 Results

5.1 Detection by Each Layer

Table 7 shows what each layer detects on the generated dataset, in the unit each layer naturally works in: the DPI engine judges individual requests, the flow monitor judges one source over one second, and the LSTM judges ten-second windows. For the LSTM, a window counts as an attack window if it contains any attack second, which is why each attack class has 54 windows (45 during the attack and 9 that overlap its end).

Table 7: Detections by each layer in its own unit of analysis

Class	DPI (requests)	Flow monitor (source-seconds)	LSTM (10-s windows)
False data injection	45 / 45	0 alerts	54 / 54
Command injection	45 / 45	0 alerts	54 / 54
Flooding (DoS)	0 / 1,800	45 / 45	0 / 54
Replay	45 / 45	0 alerts	0 / 54
Stealth manipulation	0 / 45	0 alerts	54 / 54
Normal (false positives)	0 / 628	0	0 / 231

Each layer has a blind spot that another covers. Flooding consists of valid reads and leaves the process unchanged, so only the flow monitor sees it. The stealth attack uses a valid command from the authorised address, so only the LSTM sees it. Replay re-sends the controller's most recent command, which the controller wants anyway, so the process is unaffected and only the DPI engine sees it, because the request comes from an unauthorised address.

5.2 Ablation

Table 8 and Figure 5 compare the layers per second. No single layer flags more than 60% of the 225 attack seconds. Combined, the three layers flag every attack second, with a precision of 0.893.

Table 8: Per-second ablation on the simulator dataset

Layer	Precision	Recall	FPR	F1	TP / FP / FN / TN
DPI (rule)	1.000	0.600	0.000	0.750	135 / 0 / 90 / 285
Flow monitor	1.000	0.200	0.000	0.333	45 / 0 / 180 / 285
LSTM	0.833	0.600	0.095	0.698	135 / 27 / 90 / 258
Combined	0.893	1.000	0.095	0.943	225 / 27 / 0 / 258

All 27 false-positive seconds are LSTM alerts in the one to nine seconds immediately after the false-data-injection, command-injection and stealth attacks end (nine after each). A ten-second window that ends up to nine seconds after an attack still contains attack seconds, but the per-second ground truth labels those seconds normal. The LSTM raises no alert anywhere else in normal operation, and the DPI engine and flow monitor raise none at all.

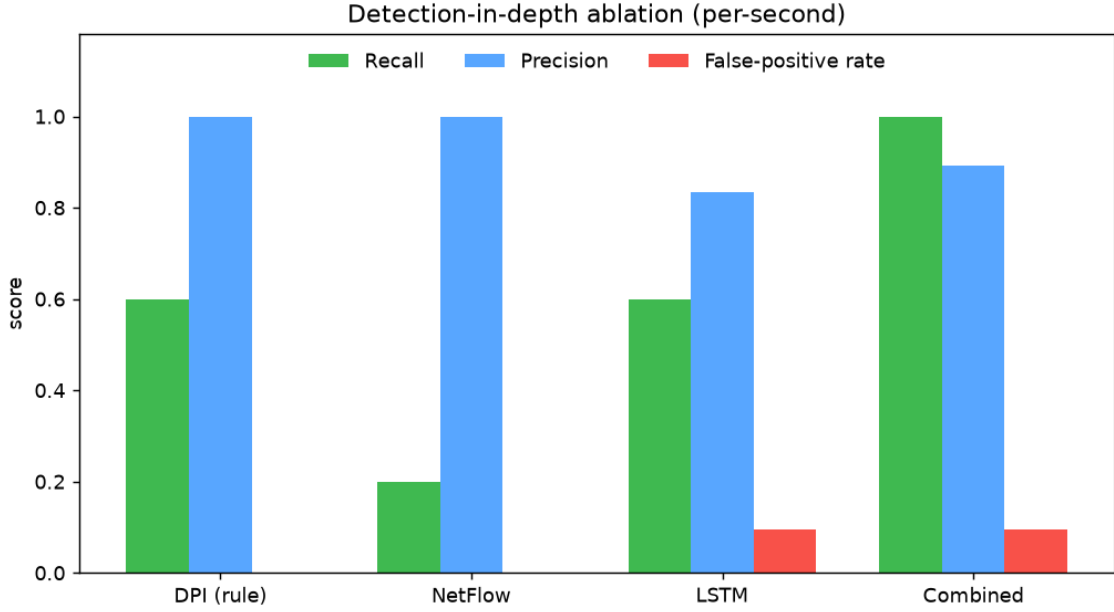


Figure 5: Per-second precision, recall and false-positive rate of each layer and of the combined system

5.3 Recall per Attack Class

Table 9 and Figure 6 break recall down by attack class. Only the combined system reaches 100% on every class.

Table 9: Per-second recall by attack class

Layer	FDI	Command inj.	Flooding	Replay	Stealth
DPI (rule)	100%	100%	0%	100%	0%
Flow monitor	0%	0%	100%	0%	0%
LSTM	100%	100%	0%	0%	100%
Combined	100%	100%	100%	100%	100%

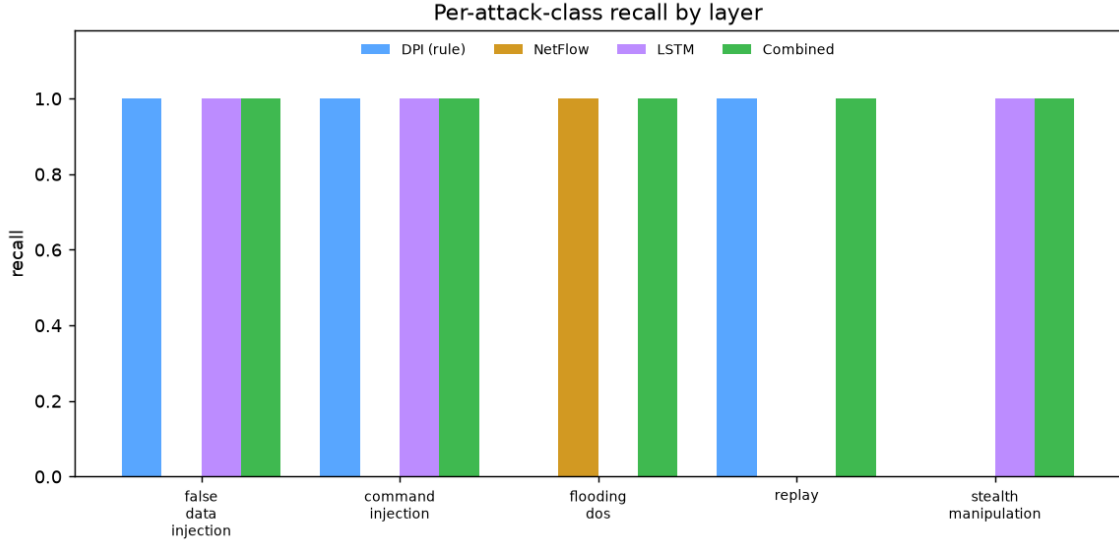


Figure 6: Recall per attack class for each layer

5.4 Detection Latency

Table 10: Seconds from the start of each attack to the first alert

Layer	FDI	Command inj.	Flooding	Replay	Stealth
DPI (rule)	0	0	missed	0	missed
Flow monitor	missed	missed	0	missed	missed
LSTM	0	0	missed	missed	0
Combined	0	0	0	0	0

The combined system raises an alert in the first second of every attack. The detectors run over the recorded logs rather than as live services; the DPI engine works on individual requests and the flow monitor on one-second windows, so both could run as traffic arrives, while the LSTM needs a full ten-second window.

5.5 External Validation on HAI 22.04

The simulator results show the design working on attacks and conditions the team designed. To test whether the learning-based layer generalises, the same LSTM autoencoder was trained and tested on HAI 22.04 [3], [4], data from a real hardware-in-the-loop testbed with boiler, turbine, water-treatment and HIL processes, sampled at one reading per second. HAI provides process data only, not network traffic, so the DPI engine and flow monitor could not be evaluated on it. Table 11 lists the files used.

Table 11: HAI 22.04 files used

File	Role	Duration	Attacks
train1.csv	Training (rounds 1 and 2)	26.0 h	none
train2.csv	Training (round 2)	56.0 h	none
train3.csv	Training (round 2)	35.0 h	none
train4.csv	Threshold calibration only (round 2)	24.0 h	none
test1.csv	Test	24.0 h	885 s in 7 episodes (~1%)

Two configurations were planned in advance and each was run once. **Round 1** trained on train1 and set the threshold at the 99.9th percentile of the training scores. **Round 2** trained on train1 to train3 and set the threshold at the 99.9th percentile of scores on train4, a separate normal recording the model never trained on. The model, window length and threshold rule were identical in both rounds. Sensors that never varied during training were dropped, leaving 60 signals in round 1 and 69 in round 2, and standardised inputs were clipped to plus or minus five standard deviations (see below). Table 12 gives the results for both scoring methods.

Table 12: LSTM results on HAI test1 (window level; random PR-AUC = 0.011)

Metric	R1 mean MSE	R1 worst signal	R2 mean MSE	R2 worst signal
ROC-AUC	0.742	0.742	0.586	0.465
PR-AUC	0.081	0.044	0.055	0.010
Precision (deployable threshold)	0.012	0.012	0.259	0.000
Recall (deployable threshold)	0.978	0.998	0.045	0.000
F1 (deployable threshold)	0.023	0.023	0.077	0.000
False-positive rate	0.914	0.940	0.001	0.002
Attack episodes detected	7 / 7 *	7 / 7 *	2 / 7	0 / 7
Best F1 with threshold chosen on test labels	0.102	0.057	0.093	0.025

* In round 1 the detector flags more than 90% of the test day, so detecting all seven episodes is not a meaningful result.

Round 1

The first attempt, before any input clipping, scored ROC-AUC 0.47, no better than chance. The cause was a single sensor, P1_PCV02Z, whose standard deviation during training was 0.0037 but whose normal values on the test day lay a median of about 127 standard

deviations from the training mean. That one sensor dominated every window's error. Clipping standardised inputs to plus or minus five standard deviations, a conventional outlier bound chosen before re-running, raised ROC-AUC to 0.74. The score now ranked attack windows above normal ones, but the threshold taken from the training data lay below almost the entire test day (Figure 7), so the detector alarmed more than 90% of the time.

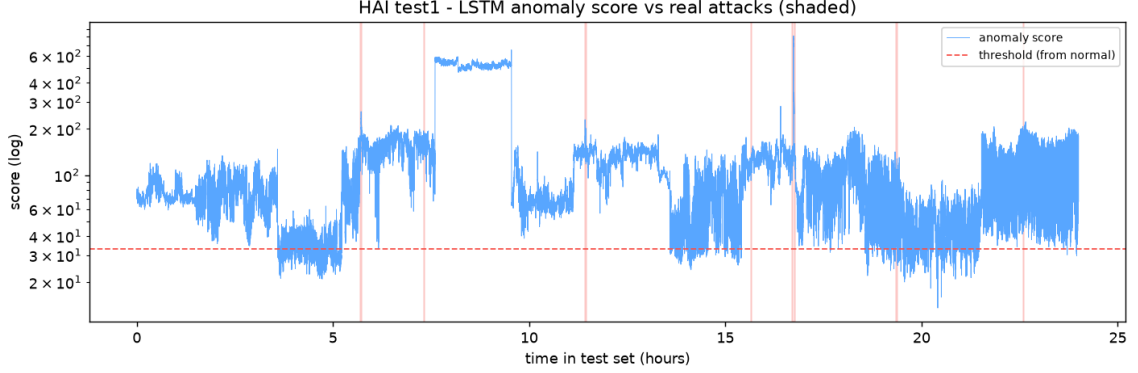


Figure 7: HAI round 1: worst-signal anomaly score over the test day, with attacks shaded; the threshold from training data lies below most of the day

Round 2

Setting the threshold on a separate normal recording fixed the false alarms: the false-positive rate fell from about 0.91 to 0.001. With plain MSE the detector then caught 2 of the 7 attack episodes, with 26% precision and a median delay of 45 seconds. Its ability to separate attacks from normal operation fell, however (ROC-AUC 0.59 with plain MSE and 0.47 with the worst-signal score). Figure 8 shows why. The score stays high for the first 16.5 hours of the test day and then drops by a factor of about 50 at about 16.8 hours, with no attack causing it: the plant was running in an operating mode that appears in none of the training recordings or the calibration recording. The score follows these mode changes rather than the attacks, which are short events on top of them. Training on three recordings also widened the standardisation ranges, so a given deviation became smaller in standardised units.

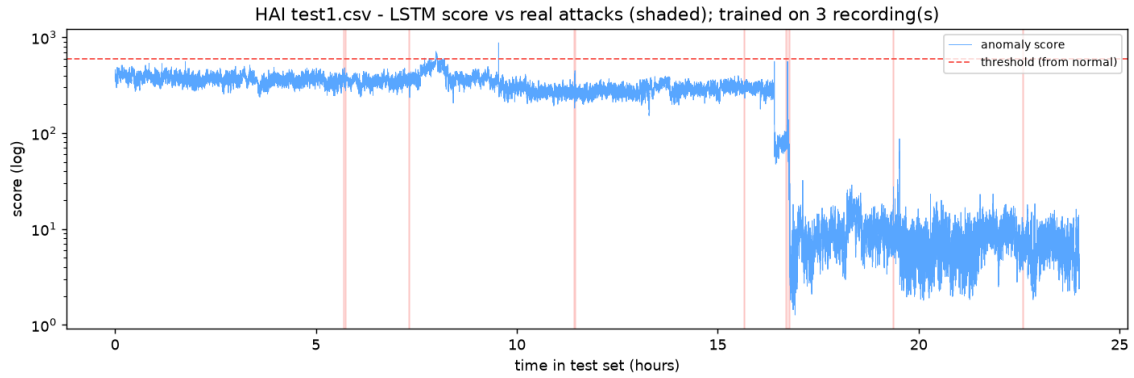


Figure 8: HAI round 2: worst-signal score after training on three recordings; the large drop near 16.8 h is a change of operating mode, not an attack

The worst-signal score was fragile on this data. It divides each sensor's error by the spread of that error during training; for sensors the model reconstructs almost perfectly, the spread is tiny, so a mode change in one of them produces an enormous score, undoing the effect of the input clipping. The same score worked on the simulator, where operating conditions never change and the stealth attack affects a single signal.

Tuning stopped after round 2. Repeatedly adjusting the model and re-checking against the labelled test file would fit the model to that file and make the validation meaningless.

6 Discussion and Limitations

6.1 What the Results Show

On the simulator, the three layers behave as designed and complement each other: each misses at least two attack types on its own, and together they detect all five in the first second. The DPI engine gives the kind of alert an operator can act on immediately, naming the source, the device and the value; the flow monitor catches a flood that no content-based check would see; and the LSTM catches a manipulation that is valid at the protocol level. The ledger reliably detects any change to recorded evidence.

The HAI validation shows that the learning-based layer, as designed, does not transfer to a real plant whose operating mode changes from one recording to the next. With a threshold learned from the training data it raises constant false alarms; with a threshold calibrated on a separate normal recording it misses most attacks. Handling operating-mode changes is the main open problem for this layer.

6.2 Limitations

- **Simulator results are optimistic.** The attacks and operating conditions were designed by the team, the dataset is a single 510-second run with one random seed, and the detectors were tested on data from the same generator they were developed with.
- **The LSTM does not generalise to HAI** (Section 5.5), and its ten-second window produces alerts for up to nine seconds after an attack ends.
- **The replay scenario is weak.** It re-sends the controller's most recent command, which has no effect on the process. Replaying an old, out-of-context command would disturb the process and has not been tested.
- **The DPI engine relies on correct configuration.** It needs an accurate register map and authorised-writer list, and an attacker who controls the authorised HMI bypasses its source check. It decodes Modbus only; ENIP/CIP decoding was not implemented.
- **The flow monitor is a per-source rate threshold.** A flood spread across many sources, or a slow flood, could stay under it.
- **No packet capture.** The network log is written by the traffic generator rather than decoded from captured packets, and the DPI engine and flow monitor were not evaluated on real network traffic because HAI contains none.

- **Detection runs offline.** The three layers run over recorded logs through the correlator; the live dashboard uses a safe-range check for demonstration.
- **The ledger is a signed hash chain on one host.** An attacker who controlled that host and its signing key could rewrite the chain; Hyperledger Fabric or external anchoring of the latest hash was not implemented.
- **Components prepared but not used.** OpenPLC and PostgreSQL are defined in the project's Docker configuration but were not used, and the proposal's multi-point coordinated attack was not implemented as a separate scenario.

6.3 Lessons Learned

- **Plot generated data before trusting results built on it.** The controller error in Section 4.3 was invisible in the summary numbers and obvious in a plot.
- **Design choices that work on clean simulations can fail on real data.** The worst-signal score was essential on the simulator and harmful on HAI.
- **Plan external validation runs in advance and stop tuning.** Otherwise the test set becomes a training set.

7 Conclusion and Future Work

7.1 Conclusion

This project built, without hardware, a detection-in-depth framework for a simulated water treatment plant that communicates over real Modbus TCP. A protocol-aware rule engine, a flow-volume monitor and an LSTM autoencoder each detect attacks the others miss, and together detect all five implemented attack types in the first second, with all false positives confined to the nine seconds after an attack. A signed hash-chain ledger makes every recorded command and alert tamper-evident, and an operator dashboard shows plant state, alerts and ledger integrity. Validation on the public HAI dataset showed that the learning-based layer does not yet cope with a real plant's changing operating modes, which is the most important direction for further work.

7.2 Status of the Objectives

Table 13: Objectives from Section 1.3 and their status

#	Objective	Status
1	Simulate a multi-stage water treatment process	Met, with a Python process model and controller (not Simulink or OpenPLC)
2	Generate traffic and logs for normal operation and attacks	Met for five attack types; the network log is generator-recorded, not a packet capture
3	Protocol-aware rule-based detector	Met for Modbus
4	LSTM anomaly detector	Met on the simulator; weak on the public HAI dataset
5	Tamper-evident ledger	Met with a signed hash chain; Hyperledger Fabric not implemented
6	Operator dashboard	Met (live plant view, demonstration alerts, ledger integrity); PostgreSQL not used
7	Evaluate layers and test generalisation	Met: per-layer ablation on the simulator and two validation rounds on HAI

7.3 Future Work

- Make the LSTM layer robust to operating-mode changes, for example by detecting the plant's mode and normalising within it, or by scoring deviations from a rolling baseline of the last few minutes.
- Train on all six HAI training recordings, confirm any improvement on the independent test2 file, and report HAI's official eTaPR metric for comparison with published results.

- Capture the Modbus traffic with tshark and run the DPI engine on decoded packets; add an ENIP/CIP decoder.
- Run the three detectors as streaming services feeding the live dashboard, with alerts stored in a database.
- Add a Hyperledger Fabric backend, or anchor the latest ledger hash externally at regular intervals.
- Put OpenPLC in the loop, and generate larger datasets with several seeds, varied operating conditions, multi-point attacks and replays of out-of-context commands.

References

- [1] M. R. Gauthama Raman, S. Khandekar, R. Murarishetti, C. Z. Y. Caven, N. G. F. Eric and J. Zhou, “Network-based real-time detection of data manipulation attacks in industrial control systems,” *International Journal of Information Security*, vol. 25, art. 12, 2026.
- [2] Z. Jadidi, E. Foo, M. Hussain and C. Fidge, “Automated detection-in-depth in industrial control systems,” *The International Journal of Advanced Manufacturing Technology*, vol. 118, pp. 2467–2479, 2022.
- [3] H.-K. Shin, W. Lee, J.-H. Yun and B.-G. Min, “HAI 1.0: HIL-based augmented ICS security dataset,” in *Proc. 13th USENIX Workshop on Cyber Security Experimentation and Test (CSET)*, 2020.
- [4] HAI security dataset, version 22.04. [Online]. Available: <https://github.com/icsdataset/hai>
- [5] Modbus Organization, “MODBUS Application Protocol Specification V1.1b3,” 2012.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] J. Goh, S. Adep, K. N. Junejo and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Proc. International Conference on Critical Information Infrastructures Security (CRITIS)*, 2016.
- [8] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal and G. Shroff, “LSTM-based encoder-decoder for multi-sensor anomaly detection,” *arXiv:1607.00148*, 2016.
- [9] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
- [11] PyModbus documentation. [Online]. Available: <https://pymodbus.readthedocs.io>

Appendix A Reproducing the Results

From the repository root, with the Python environment installed from requirements.txt:

```
# simulator dataset, detectors and evaluation
python -m net.generate_dataset --out-dir data/run1 --speedup 0 --seed 42
python -m detect.lstm.train --device-log data/run1/device_log.csv
python -m detect.correlator.run      # coverage, ledger, tamper demo
python -m eval.evaluate              # ablation, recall, latency, plots

# HAI validation (CSV files downloaded into data/public/hai/)
python -m eval.hai_validate
python -m eval.hai_validate \
    --train
data/public/hai/train1.csv,data/public/hai/train2.csv,data/public/hai/train3.csv \
    --calib data/public/hai/train4.csv --tag multiday

# report figures, dashboard and tests
python -m report.make_figures
python -m app.dashboard              # http://127.0.0.1:8000
python -m pytest -q
```

Appendix B Register Map

Sensor ranges are the safe operating ranges used by the dashboard and the DPI engine; actuator entries list their legal states (0 = off/closed, 1 = on/open, 2 = in transition).

Table 14: Plant register map

Tag	Addr.	Type	Stage	Unit	Range / states	Description
LIT101	0	sensor	Intake	mm	250 – 1100	Raw water tank level
FIT101	1	sensor	Intake	m ³ /h	0 – 3	Raw water inflow rate
MV101	5	actuator	Intake	state	0 / 1 / 2	Motorised inlet valve
P101	6	actuator	Intake	state	0 / 1	Raw water transfer pump
AIT201	10	sensor	Chemical dosing	uS/cm	100 – 300	Conductivity after dosing
AIT202	11	sensor	Chemical dosing	pH	6.5 – 8.5	pH after chemical dosing
FIT201	12	sensor	Chemical dosing	m ³ /h	0 – 3	Dosing stage flow
P201	15	actuator	Chemical dosing	state	0 / 1	NaOCl dosing pump
P203	16	actuator	Chemical dosing	state	0 / 1	HCl dosing pump
DPIT301	20	sensor	Ultrafiltration	kPa	0 – 40	UF differential pressure
LIT301	21	sensor	Ultrafiltration	mm	250 – 1100	UF feed tank level
FIT301	22	sensor	Ultrafiltration	m ³ /h	0 – 3	UF permeate flow
MV301	25	actuator	Ultrafiltration	state	0 / 1 / 2	UF backwash valve
P301	26	actuator	Ultrafiltration	state	0 / 1	UF feed pump
AIT401	30	sensor	Dechlorination	ppm	0 – 1	Residual chlorine
LIT401	31	sensor	Dechlorination	mm	250 – 1100	RO feed tank level
P401	35	actuator	Dechlorination	state	0 / 1	UV dechlorinator / feed pump
PIT501	40	sensor	Reverse osmosis	kPa	0 – 300	RO feed pressure
FIT501	41	sensor	Reverse osmosis	m ³ /h	0 – 2	RO permeate flow
AIT501	42	sensor	Reverse osmosis	uS/cm	5 – 100	RO permeate conductivity
P501	45	actuator	Reverse osmosis	state	0 / 1	RO high-pressure pump
MV501	46	actuator	Reverse osmosis	state	0 / 1 / 2	RO permeate valve

Appendix C Automated Tests

Table 15: Automated tests

Test file	Tests	What it checks
test_modbus_roundtrip.py	1	The Modbus server serves live plant values and accepts writes
test_dataset_generator.py	2	Log schemas, attack windows present, spoofed values, actuators restored after attacks
test_dpi_engine.py	6	Each rule fires on its attack; legitimate writes, reads and the compromised-HMI command are not flagged
test_netflow_monitor.py	2	Floods are flagged and attributed to the attacker; no alerts on normal traffic
test_lstm_pipeline.py	1	A forced-on actuator scores above every normal window
test_ledger.py	6	Intact chain verifies; edits, deletions, reordering and forged signatures are detected; persistence
test_evaluate.py	3	Confusion counts, per-class recall, episode extraction and latency
test_hai_helpers.py	3	Episode extraction, threshold metrics and episode detection for the HAI script